DS HOMEWORK PROGRAMS EXERCISE-4

1.Implement a Stack using Queue data structure.

```
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front = 0, rear = -1;
void push(int value)
{
   int i;
   if (rear == MAX - 1)
   {
      printf("Stack Overflow\n");
      return;
   }
   for (i = rear; i >= front; i--)
      queue[i + 1] = queue[i];
   queue[front] = value;
   rear++;
}
void pop()
{
   if (rear < front)
   {
      printf("Stack Underflow\n");
      return;
  }
   printf("Popped element: %d\n", queue[front]);
   front++;
}
void display()
{
```

```
    int i;
    if (rear < front)
    {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack elements: ");
    for (i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}
int main()
{
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    push(40);
    display();
    pop();
    display();
    return 0;
}
```

OUTPUT

```
Stack elements: 30 20 10
Popped element: 30
Stack elements: 20 10
Stack elements: 40 20 10
Popped element: 40
```

Stack elements: 20 10

2. Implement a Queue using Stack data structure.

```
#include <stdio.h>
#define MAX 5
int stack1[MAX],stack2[MAX];
int top1=-1,top2=-1;
void enqueue(int value){
   if(top1==MAX-1){
      printf("Queue Overflow\n");
      return;
   }
   stack1[++top1]=value;
}
void dequeue(){
   int i;
   if(top1==-1 && top2==-1){
      printf("Queue Underflow\n");
      return;
   }
   if(top2==-1){
      while(top1!=-1)
         stack2[++top2]=stack1[top1--];
   }
   printf("Dequeued element: %d\n",stack2[top2--]);
}
void display(){
   int i;
   if(top1==-1 && top2==-1){
      printf("Queue is empty\n");
      return;
```

```
    }
    printf("Queue elements: ");
    for(i=top2;i>=0;i--)
        printf("%d ",stack2[i]);
    for(i=0;i<=top1;i++)
        printf("%d ",stack1[i]);
    printf("\n");
}
int main(){
    enqueue(10);
    enqueue(20);
    enqueue(30);
    display();
    dequeue();
    display();
    enqueue(40);
    display();
    dequeue();
    display();
    return 0;
}
```

OUTPUT

```
Queue elements: 10 20 30
Dequeued element: 10
Queue elements: 20 30
Queue elements: 20 30 40
Dequeued element: 20
Queue elements: 30 40
```

3. Write a C program using Stack data structure to implement managing the history of web pages visited.

```
#include <stdio.h>
#include <string.h>
#define MAX 5
char history[MAX][100];
int top=-1;
void visit(char page[]){
   if(top==MAX-1){
      printf("History is full\n");
      return;
   }
   strcpy(history[++top],page);
   printf("Visited: %s\n",page);
}
void back(){
   if(top==-1){
      printf("No history available\n");
      return;
   }
   printf("Going back from: %s\n",history[top--]);
}
void display(){
   int i;
   if(top==-1){
      printf("History is empty\n");
      return;
   }
   printf("Browser History:\n");
   for(i=top;i>=0;i--)
      printf("%s\n",history[i]);
}
int main(){
```

```
    visit("Google");
    visit("YouTube");
    visit("GitHub");
    display();
    back();
    display();
    return 0;
}
```

OUTPUT

```
Visited: Google
Visited: YouTube
Visited: GitHub
Browser History:
GitHub
YouTube
Google
Going back from: GitHub
Browser History:
YouTube
Google
```

4. Write a C program to sort a Stack using a temporary Stack.

```
#include <stdio.h>
#define MAX 5
int stack[MAX],temp[MAX];
int top=-1,ttop=-1;
void push(int value){
    stack[++top]=value;
}
void sort(){
    int value;
```

```
    while(top!=-1){
        value=stack[top--];
        while(ttop!=-1 && temp[ttop]>value)
            stack[++top]=temp[ttop--];
        temp[++ttop]=value;
    }
    while(ttop!=-1)
        stack[++top]=temp[ttop--];
}
void display(){
    int i;
    printf("Sorted Stack: ");
    for(i=top;i>=0;i--)
        printf("%d ",stack[i]);
    printf("\n");
}
int main(){
    push(30);
    push(10);
    push(50);
    push(20);
    push(40);
    printf("Original Stack: ");
    display();
    sort();
    display();
    return 0;
}
```

OUTPUT

Original Stack: 40 20 50 10 30

Sorted Stack: 10 20 30 40 50

5. The Stock Span Problem is a financial problem where we have a series of daily price quotes for a stock denoted by an array and the task is to calculate the span of the stock's price for all days. The span of a stock's price for the ith day gives the maximum number of consecutive days leading up to the ith day (including current day) where stock's price is less than or equal to its price on day i.

Eg:
I/p: [100, 80, 60, 70, 60, 75, 85]
O/p: [1, 1, 1, 2, 1, 4, 6]

```
#include <stdio.h>

#define MAX 100

int main()

{

   int price[MAX], span[MAX], stack[MAX];

   int n, top = -1;

   int i;

   printf("Enter number of days: ");

   scanf("%d", &n);

   printf("Enter stock prices: ");

   for (i = 0; i < n; i++)

      scanf("%d", &price[i]);

   for (i = 0; i < n; i++)

   {

      while (top != -1 && price[stack[top]] <= price[i])

         top--;

      if (top == -1)

         span[i] = i + 1;

      else

         span[i] = i - stack[top];

      stack[++top] = i;

   }

   printf("Stock Span: ");

   for (i = 0; i < n; i++)

      printf("%d ", span[i]);

   return 0;
```

```
}
```

OUTPUT

```
Enter number of days: 7
Enter stock prices: 100 80 60 70 60 75 85
Stock Span: 1 1 1 2 1 4 6
```

6. Given an array, for each element find the nearest element to its right that has a higher frequency than the current element. If no such element exists, set value to -1. Implement using stack.

```
#include <stdio.h>
#define MAX 100
int main()
{
   int arr[MAX], freq[MAX], stack[MAX], result[MAX];
   int n, top = -1, i, j;
   printf("Enter number of elements: ");
   scanf("%d", &n);
   printf("Enter array elements: ");
   for (i = 0; i < n; i++)
      scanf("%d", &arr[i]);
   for (i = 0; i < n; i++)
   {
      freq[i] = 0;
      for (j = 0; j < n; j++)
      {
         if (arr[i] == arr[j])
            freq[i]++;
      }
   }
   for (i = n - 1; i >= 0; i--)
   {
      while (top != -1 && freq[stack[top]] <= freq[i])
```

```
            top--;
        if (top == -1)
            result[i] = -1;
        else
            result[i] = arr[stack[top]];
        stack[++top] = i;
    }
    printf("Result: ");
    for (i = 0; i < n; i++)
        printf("%d ", result[i]);
    return 0;
}
```

OUTPUT

```
Enter number of elements: 7
Enter array elements: 4 5 4 6 5 5 7
Result: 5 5 5 5 5 -1 -1
```

7. Given a pattern containing only 'I's and 'D's, 'I' for Increasing and 'D' for Decreasing. Devise an algorithm using stack to print minimum number following that pattern. Digits from 1-9 and digits can't repeat.

Eg:
I/p: D O/p: 21
I/p: I D D I O/p: 121435

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[100];
    int stack[100];
    int top = -1;
    int num = 1;
    int i, n;
```

```
    printf("Enter pattern: ");
    scanf("%s", str);
    n = strlen(str);
    printf("Output: ");
    for (i = 0; i <= n; i++)
    {
        stack[++top] = num++;
        if (i == n || str[i] == 'I')
        {
            while (top != -1)
                printf("%d", stack[top--]);
        }
    }
    return 0;
}
```

Enter pattern: IDDI

Output: 13254

8. Given an array of digits (values are from 0 to 9), find the minimum possible sum of two numbers formed from digits of the array. All digits of given array must be used to form the two numbers. Use Queue for implementation.

Eg:
I/p: [6, 8, 4, 5, 2, 3]
O/p: 604 (min sum formed by numbers 358 and 246)

```
#include <stdio.h>
#define MAX 100
struct Queue
{
    int data[MAX];
    int front, rear;
};
void enqueue(struct Queue *q, int x)
```

```
{
    q->data[++q->rear] = x;
}
int dequeue(struct Queue *q)
{
    return q->data[q->front++];
}
int main()
{
    struct Queue q;
    int a[MAX];
    int n, i, j, temp;
    int num1 = 0, num2 = 0;
    q.front = 0;
    q.rear = -1;
    printf("Enter number of digits: ");
    scanf("%d", &n);
    printf("Enter digits: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (a[i] > a[j])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
```

```
    }
    for (i = 0; i < n; i++)
        enqueue(&q, a[i]);
    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
            num1 = num1 * 10 + dequeue(&q);
        else
            num2 = num2 * 10 + dequeue(&q);
    }
    printf("Minimum Sum = %d", num1 + num2);
    return 0;
}
```

OUTPUT

```
Enter number of digits: 5
Enter digits: 9
4
7
2
6
Minimum Sum = 316
```

9. Given an array and a positive integer k, find the first negative integer for each window (contiguous subarray) of size k. If a window does not contain a negative integer, then print 0 for that window. Implement using queue.

Eg:
I/p: [-8, 2, 3, -6, 1], k = 2
O/p: [-8, 0, -6, -6]

```
#include <stdio.h>
#define MAX 100
int main()
{
```

```
    int a[MAX], queue[MAX];
    int n, k;
    int front = 0, rear = -1;
    int i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter window size: ");
    scanf("%d", &k);
    printf("Output: ");
    for (i = 0; i < n; i++)
    {
        if (a[i] < 0)
            queue[++rear] = i;
        while (front <= rear && queue[front] <= i - k)
            front++;
        if (i >= k - 1)
        {
            if (front <= rear)
                printf("%d ", a[queue[front]]);
            else
                printf("0 ");
        }
    }
    return 0;
}
```

OUTPUT

```
Enter number of elements: 6
Enter elements: 4
```

-3

5

-2

7

-6

Enter window size: 3

Output: -3 -3 -2 -2

10.Given a matrix of dimension m x n where each cell in the matrix can have value 0, 1, or 2 which has the following meaning:

0: empty cell

1: cells having fresh oranges

2: cells having rotten oranges

The task is to find the minimum time required so that all oranges become rotten. A rotten orange at index (i,j) can rot other fresh oranges which are its neighbours (up, down, left and right). If impossible to rot every orange, then return -1. Use Queue for implementation.

```
#include <stdio.h>
#define MAX 100
struct Queue
{
   int row[MAX * MAX];
   int col[MAX * MAX];
   int front, rear;
};
void enqueue(struct Queue *q, int r, int c)
{
   q->rear++;
   q->row[q->rear] = r;
   q->col[q->rear] = c;
}
void dequeue(struct Queue *q, int *r, int *c)
```

```
{
    *r = q->row[q->front];
    *c = q->col[q->front];
    q->front++;
}
int main()
{
    int grid[MAX][MAX];
    struct Queue q;
    int m, n;
    int i, j;
    int r, c;
    int time = 0;
    int fresh = 0;
    int dr[] = {-1, 1, 0, 0};
    int dc[] = {0, 0, -1, 1};
    q.front = 0;
    q.rear = -1;
    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);
    printf("Enter matrix:\n");
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
        {
            scanf("%d", &grid[i][j]);
            if (grid[i][j] == 2)
                enqueue(&q, i, j);
            if (grid[i][j] == 1)
                fresh++;
        }
```

```
    }
    while (q.front <= q.rear && fresh > 0)
    {
        int size = q.rear - q.front + 1;
        for (i = 0; i < size; i++)
        {
            dequeue(&q, &r, &c);
            for (j = 0; j < 4; j++)
            {
                int newRow = r + dr[j];
                int newCol = c + dc[j];
                if (newRow >= 0 && newRow < m && newCol >= 0 && newCol < n &&
grid[newRow][newCol] == 1)
                {
                    grid[newRow][newCol] = 2;
                    fresh--;
                    enqueue(&q, newRow, newCol);
                }
            }
        }
        time++;
    }
    if (fresh == 0)
        printf("Minimum time = %d", time);
    else
        printf("All oranges cannot be rotten");
    return 0;
}
```

OUTPUT

```
Enter rows and columns: 3 3
Enter matrix:
```

2

1

0

1

1

0

0

1

1

Minimum time = 4